

Assignment 5

Hadeer Ghoneim

Verification Diploma

Design

```
1  module ALSU(A, B, cin, serial_in, red_op_A, red_op_B, opcode, bypass_A, bypass_B, clk, rst, direction, leds, out);
2  parameter INPUT_PRIORITY = "A";
3  parameter FULL_ADDER = "ON";
4  input clk, cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
5  input [2:0] opcode;
6  input signed [2:0] A, B;
7  output reg [15:0] leds;
8  output reg signed [5:0] out;
9
10 reg red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg, serial_in_reg;
11 reg signed [1:0] cin_reg;
12 reg [2:0] opcode_reg;
13 reg signed [2:0] A_reg, B_reg;
14
15 wire invalid_red_op, invalid_opcode, invalid;
16
17 //Invalid handling
18 assign invalid_red_op = (red_op_A_reg | red_op_B_reg) & (opcode_reg[1] | opcode_reg[2]);
19 assign invalid_opcode = opcode_reg[1] & opcode_reg[2];
20 assign invalid = invalid_red_op | invalid_opcode;
21
22 //Registering input signals
23 always @(posedge clk or posedge rst) begin
24     if(rst) begin
25         cin_reg <= 0;
26         red_op_B_reg <= 0;
27         red_op_A_reg <= 0;
28         bypass_B_reg <= 0;
29         bypass_A_reg <= 0;
30         direction_reg <= 0;
31         serial_in_reg <= 0;
32         opcode_reg <= 0;
33         A_reg <= 0;
34         B_reg <= 0;
35     end else begin
36         cin_reg <= cin;
```

```

37     red_op_B_reg <= red_op_B;
38     red_op_A_reg <= red_op_A;
39     bypass_B_reg <= bypass_B;
40     bypass_A_reg <= bypass_A;
41     direction_reg <= direction;
42     serial_in_reg <= serial_in;
43     opcode_reg <= opcode;
44     A_reg <= A;
45     B_reg <= B;
46 end
47 end
48
49 //leds output blinking
50 always @(posedge clk or posedge rst) begin
51     if(rst) begin
52         leds <= 0;
53     end else begin
54         if (invalid)
55             leds <= ~leds;
56         else
57             leds <= 0;
58     end
59 end
60
61 //ALU output processing
62 always @(posedge clk or posedge rst) begin
63     if(rst) begin
64         out <= 0;
65     end
66     else begin
67         if (bypass_A_reg && bypass_B_reg)
68             out <= (INPUT_PRIORITY == "A")? A_reg: B_reg;
69         else if (bypass_A_reg)

```

```

69     else if (bypass_A_reg)
70         out <= A_reg;
71     else if (bypass_B_reg)
72         out <= B_reg;
73     else if (invalid)
74         out <= 0;
75     else begin
76         case (opcode_reg) //edited design error
77             3'h0: begin
78                 if (red_op_A_reg && red_op_B_reg)
79                     out <= (INPUT_PRIORITY == "A")? |A_reg: |B_reg;
80                 else if (red_op_A_reg)
81                     out <= |A_reg;
82                 else if (red_op_B_reg)
83                     out <= |B_reg;
84                 else
85                     out <= A_reg | B_reg;
86             end
87             3'h1: begin
88                 if (red_op_A_reg && red_op_B_reg)
89                     out <= (INPUT_PRIORITY == "A")? ^A_reg: ^B_reg;
90                 else if (red_op_A_reg)
91                     out <= ^A_reg;
92                 else if (red_op_B_reg)
93                     out <= ^B_reg;
94                 else
95                     out <= A_reg ^ B_reg;
96             end
97             3'h2: out <= (FULL_ADDER == "ON") ? (A_reg + B_reg + cin_reg) : (A_reg + B_reg); //edited design error
98             3'h3: out <= A_reg * B_reg;
99             3'h4: begin
100                 if (direction_reg)
101                     out <= {out[4:0], serial_in_reg};

```

```

100         if (direction_reg)
101             out <= {out[4:0], serial_in_reg};
102         else
103             out <= {serial_in_reg, out[5:1]};
104     end
105     3'h5: begin
106         if (direction_reg)
107             out <= {out[4:0], out[5]};
108         else
109             out <= {out[0], out[5:1]};
110     end
111     default: out <= 0; //edit for 100% coverage and exclude it
112
113 endcase
114 end
115 end
116 end
117
118 endmodule

```

Design Interface

```
8  interface ALSU_if (clk);
9  input clk;
10 logic cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
11 logic [2:0] opcode;
12 logic signed [2:0] A, B;
13 logic [15:0] leds;
14 logic signed [5:0] out;
15 endinterface : ALSU_if
```

1. top module

```
7
8  import ALSU_test_pkg::*;
9  import uvm_pkg::*;
10 `include "uvm_macros.svh"
11
12 module top();
13
14     bit clk, rst;
15
16     initial begin
17         forever
18             #1 clk = ~clk;
19     end
20
21     ALSU_if ALSUif (clk);
22     ALSU DUT (ALSUif.A, ALSUif.B, ALSUif.cin, ALSUif.serial_in, ALSUif.red_op_A,
23     ALSUif.red_op_B, ALSUif.opcode, ALSUif.bypass_A, ALSUif.bypass_B,
24     ALSUif.clk, ALSUif.rst, ALSUif.direction, ALSUif.leds, ALSUif.out);
25     initial begin
26         uvm_config_db#(virtual ALSU_if)::set(null, "uvm_test_top", "ALSU_if", ALSUif);
27         run_test("ALSU_test");
28     end
29
30 endmodule
31
```

2. alsu_test

```
8 package ALSU_test_pkg;
9
10 import ALSU_config_pkg::*;
11 import ALSU_env_pkg::*;
12
13 import uvm_pkg::*;
14 `include "uvm_macros.svh"
15
16 class ALSU_test extends uvm_test;
17     `uvm_component_utils(ALSU_test)
18
19     ALSU_env env;
20     ALSU_config ALSU_cfg;
21
22     function new(string name = "ALSU_test", uvm_component parent = null);
23         super.new(name, parent);
24     endfunction
25
26     function void build_phase(uvm_phase phase);
27         super.build_phase(phase);
28         env = ALSU_env::type_id::create("env", this);
29         ALSU_cfg = ALSU_config::type_id::create("ALSU_cfg");
30
31         if (!uvm_config_db #(virtual ALSU_if)::get(this, "", "ALSU_if", ALSU_cfg.ALSU))
32             `uvm_fatal("build_phase", "Test - Unable to get the virtual interface");
33
34         uvm_config_db #(ALSU_config)::set(this, "", "CFG", ALSU_cfg);
35     endfunction
36
37     task run_phase(uvm_phase phase);
38         super.run_phase(phase);
39         phase.raise_objection(this);
40         □ #100; `uvm_info("run_phase", "Inside the ALSU test using", UVM_MEDIUM)
41         phase.drop_objection(this);
42     endtask: run_phase
43
44 endclass
45 endpackage
46
```

3. alsu_env

```
8 package ALSU_env_pkg;
9
10 import ALSU_driver_pkg::*;
11 import uvm_pkg::*;
12 `include "uvm_macros.svh"
13
14 class ALSU_env extends uvm_env;
15
16   `uvm_component_utils(ALSU_env)
17   ALSU_driver driver;
18
19   function new(string name = "ALSU_env", uvm_component parent = null);
20     super.new(name, parent);
21   endfunction
22
23   function void build_phase(uvm_phase phase);
24     super.build_phase(phase);
25     driver = ALSU_driver::type_id::create("driver", this);
26   endfunction: build_phase
27
28   endclass : ALSU_env
29 endpackage : ALSU_env_pkg
```

4. alsu_driver

```
1 package ALSU_driver_pkg;
2
3 import ALSU_config_pkg::*;
4 import uvm_pkg::*;
5 `include "uvm_macros.svh"
6
7 class ALSU_driver extends uvm_driver;
8   `uvm_component_utils(ALSU_driver)
9
10   virtual ALSU_if ALSU;
11   ALSU_config ALSU_cfg;
12
13   function new(string name = "ALSU_driver", uvm_component parent = null);
14     super.new(name, parent);
15   endfunction
16
17   function void build_phase(uvm_phase phase);
18     super.build_phase(phase);
19     if (!uvm_config_db #(ALSU_config)::get(this, "", "CFG", ALSU_cfg))
20       `uvm_fatal("build_phase", "Unable to get configuration object");
21   endfunction
22
23   function void connect_phase(uvm_phase phase);
24     super.connect_phase(phase);
25     ALSU = ALSU_cfg.ALSU;
26   endfunction
27
28   task run_phase(uvm_phase phase);
29
```

```

27
28     task run_phase(uvm_phase phase);
29
30         super.run_phase(phase);
31         ALSU.A = 0; ALSU.B = 0; ALSU.cin = 0; ALSU.serial_in = 0; ALSU.red_op_A = 0;
32         ALSU.red_op_B = 0; ALSU.opcode = 0; ALSU.bypass_A = 0; ALSU.bypass_B = 0;
33         ALSU.direction = 0; ALSU.rst = 1;
34         @(negedge ALSU.clk); ALSU.rst = 0;
35
36         forever begin
37
38             @(negedge ALSU.clk);
39             ALSU.A = $random; ALSU.B = $random; ALSU.cin = $random;
40             ALSU.serial_in = $random; ALSU.red_op_A = $random;
41             ALSU.red_op_B = $random; ALSU.opcode = $random;
42             ALSU.bypass_A = $random; ALSU.bypass_B = $random;
43             ALSU.direction = $random;
44
45         end
46     endtask
47
48 endclass
49
50 endpackage

```

5. alsu_config_obj

```

1  package ALSU_config_pkg;
2
3      import uvm_pkg::*;
4      `include "uvm_macros.svh"
5
6      class ALSU_config extends uvm_object;
7          `uvm_object_utils(ALSU_config)
8
9          virtual ALSU_if ALSU;
10
11          function new(string name = "ALSU_config");
12              super.new(name);
13          endfunction
14
15      endclass
16  endpackage

```

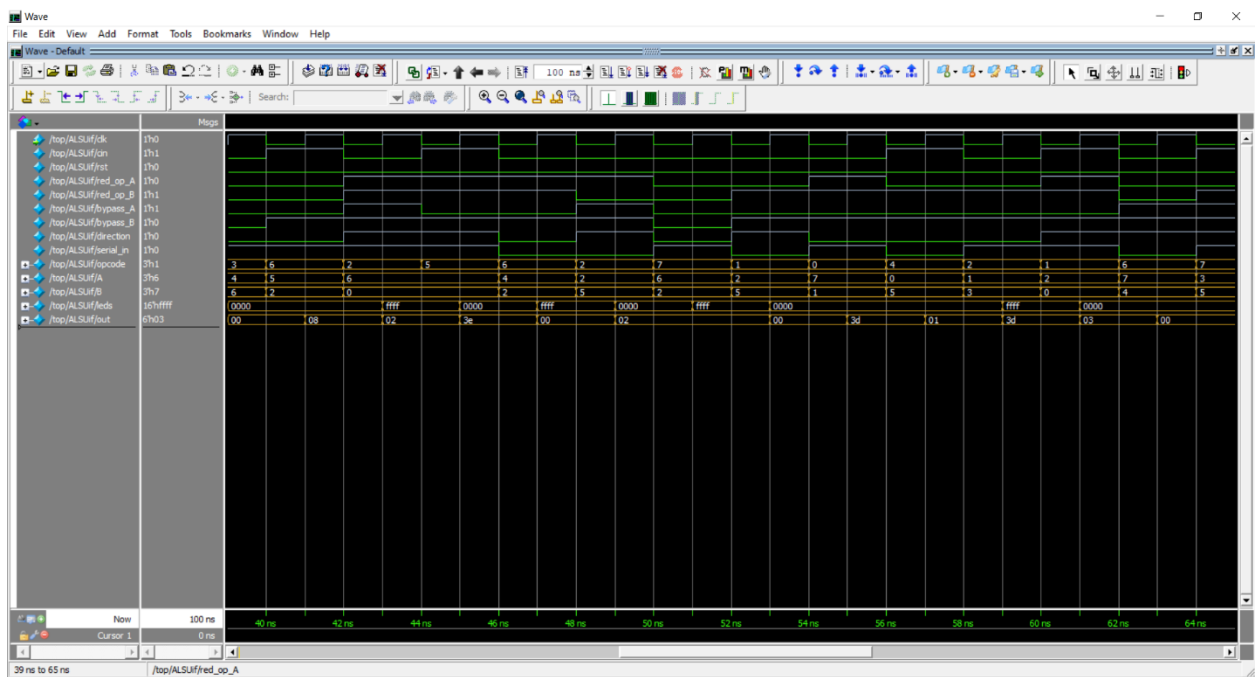
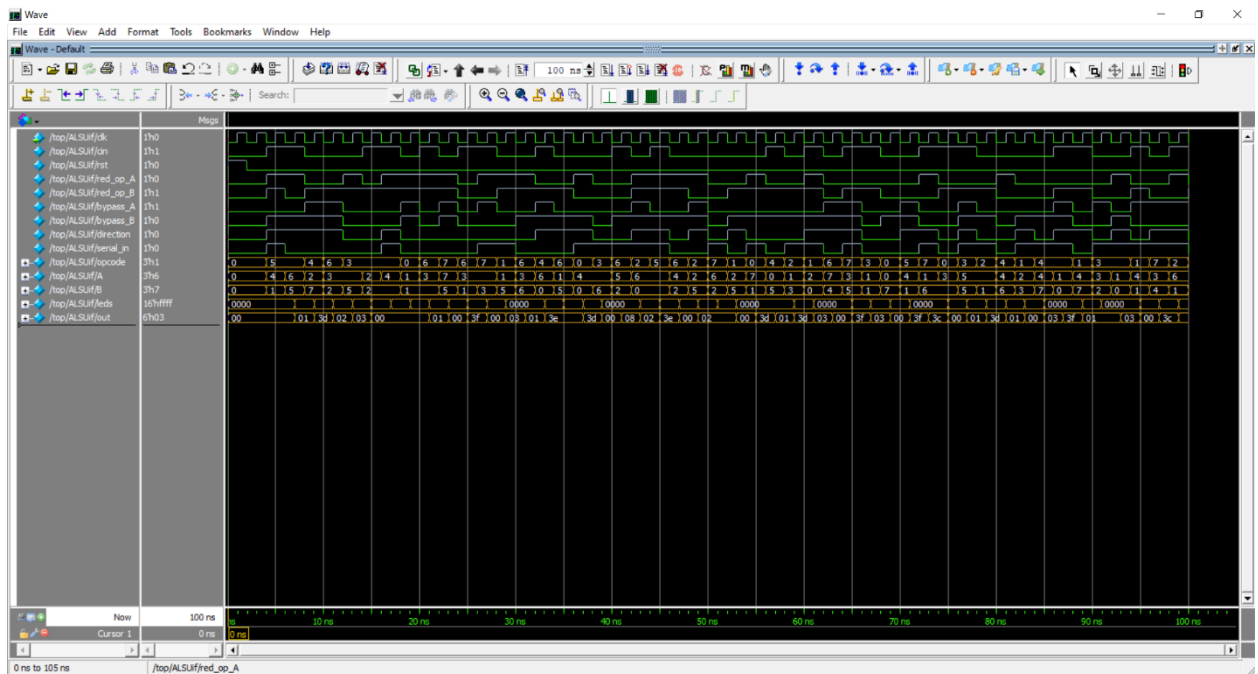

6. list file

```
1  ALSU_if.sv
2  ALSU.v
3  ALSU_config.sv
4  ALSU_driver.sv
5  ALSU_env.sv
6  ALSU_test.sv
7  top.sv
```

7.do file

```
1  vlib work
2  vlog -f src_files.list
3  vsim -voptargs=+acc work.top -classdebug -uvmcontrol=all
4  add wave /top/ALSUif/*
5  run -all
```

8. waveform



9. transcript

Transcript

File Edit View Bookmarks Window Help

```
Transcript
# Loading work.ALSU_if(fast__2)
# Loading work.ALSU(fast)
# Loading C:/Users/General/Documents/questasim64_2021.1/uvvm-1.1d/win64/uvvm_dpi.dll
#
#-----
# UVM-1.1d
# (C) 2007-2013 Mentor Graphics Corporation
# (C) 2007-2013 Cadence Design Systems, Inc.
# (C) 2006-2013 Synopsys, Inc.
# (C) 2011-2013 Cypress Semiconductor Corp.
#
#-----
#
# ***** IMPORTANT RELEASE NOTES *****
#
# You are using a version of the UVM library that has been compiled
# with `UVM_NO_DEPRECATED undefined.
# See http://www.eda.org/svdb/view.php?id=3313 for more details.
#
# You are using a version of the UVM library that has been compiled
# with `UVM_OBJECT_MUST_HAVE_CONSTRUCTOR undefined.
# See http://www.eda.org/svdb/view.php?id=3770 for more details.
#
# (Specify +UVM_NO_RELNOTES to turn off this notice)
#
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(277) @ 0: reporter [Questa UVM] QUESTA_UVM-1.2.3
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(278) @ 0: reporter [Questa UVM] questa_uvm::init(all)
# UVM_INFO @ 0: reporter [RNTST] Running test ALSU_test...
# UVM_INFO ALSU_test.sv(40) @ 100: uvm_test_top [run_phase] Inside the ALSU test using
# UVM_INFO verilog_src/uvvm-1.1d/src/base/uvvm_objection.svh(1267) @ 100: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 5
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
# ** Report counts by id
# [Questa UVM] 2
# [RNTST] 1
# [TEST_DONE] 1
# [run_phase] 1
# ** Note: $finish : C:/Users/General/Documents/questasim64_2021.1/win64/./verilog_src/uvvm-1.1d/src/base/uvvm_root.svh(430)
# Time: 100 ns Iteration: 54 Instance: /top
# 1
# Break in Task uvm_pkg/uvvm_root::run_test at C:/Users/General/Documents/questasim64_2021.1/win64/./verilog_src/uvvm-1.1d/src/base/uvvm_root.svh line 430
VSIM 4>
```